



UNIVERSITÀ
DEGLI STUDI
DI PADOVA

UNIVERSITÀ DEGLI STUDI DI PADOVA

DIPARTIMENTO DI MATEMATICA

LAUREA TRIENNALE IN INFORMATICA

TECNOLOGIE WEB

UniFix

Informazioni di Accesso e Contatto:

Utente Amministratore: username: admin, password: admin

Utente Base: username: user, password: user

Utente Tecnico: username: tecnico, password: tecnico

Indirizzo Sito: <http://tecweb.studenti.math.unipd.it/msanguin>

Referente: aldo.bettega@studenti.unipd.it

Anno Accademico 2025/2026

Indice

1. Introduzione	3
1.1. Contesto del problema	3
1.2. Soluzione	3
2. Metodologia di lavoro	3
2.1. Analisi e studio fattibilità	3
2.2. Progettazione e mockup visivo	3
2.3. Organizzazione del lavoro e gestione dei task	4
2.4. Flusso di lavoro e versioning	4
2.5. Qualità del Codice e Continuous Integration	4
3. Fasi del progetto	4
3.1. Analisi dei Requisiti	4
3.1.1. Attori del sistema	5
3.1.2. Casi d'uso principali	5
3.2. Architettura e Progettazione	5
3.2.1. Pattern MVC (Model-View-Controller)	5
3.2.2. Diagrammi delle classi	6
3.2.3. Motore di templating	7
3.2.4. Routing e sicurezza	7
3.2.5. Persistenza dati	7
3.2.5.1. Progettazione del database e integrità dei dati	7
3.2.5.2. Gestione delle dipendenze e vincoli referenziali	9
3.2.5.3. Vincoli di dominio e logica procedurale	9
4. Accessibilità	9
4.1. Struttura semantica e supporto agli screen reader	9
4.2. Accessibilità visiva e mobile-first	10
4.3. Visualizzazione dati tabellari	10
4.4. Stampa delle pagine	10
4.5. Accessibilità dei Form e Inserimento Dati	11
5. Suddivisione dei compiti	12
6. Conclusioni e Sviluppi futuri	13
6.1. Conclusioni	13
6.2. Sviluppi futuri	13

1. Introduzione

1.1. Contesto del problema

La gestione e la manutenzione delle infrastrutture universitarie, in particolare delle aule didattiche e dei laboratori, rappresenta una sfida organizzativa complessa. Spesso, la comunicazione dei guasti (come strumentazione non funzionante, problemi di illuminazione o danni agli arredi) avviene in modo frammentato o tramite canali non ottimizzati. Questo genera non solo disagi per gli studenti e i docenti che vivono gli spazi, ma rallenta anche i tempi di intervento del personale tecnico, ostacolato dalla mancanza di un sistema centralizzato per la ricezione e la gestione delle priorità. Inoltre, molti degli strumenti digitali attualmente in uso peccano di scarsa inclusività, rendendo la segnalazione dei problemi non accessibile a tutti gli utenti.

1.2. Soluzione

Per rispondere a questa esigenza nasce UniFix, una piattaforma web progettata con il duplice obiettivo di semplificare l'esposizione dei problemi da parte degli utenti e di fornire uno strumento di back-office efficiente per l'amministrazione. UniFix si propone come un ponte digitale tra chi vive gli spazi universitari e chi li gestisce, offrendo interfacce dedicate e ottimizzate per tre tipologie di attori: lo studente (utente base), il tecnico manutentore e l'amministratore di sistema.

2. Metodologia di lavoro

Lo sviluppo di UniFix ha richiesto una rigorosa organizzazione metodologica, necessaria per coordinare efficacemente il lavoro di un team operante in modalità prevalentemente asincrona, a causa della concomitanza di altri impegni accademici e professionali (esami, tirocini). Per garantire produttività costante, prevedibilità dell'andamento e un prodotto finale di alta qualità, il ciclo di vita del progetto è stato gestito applicando principi mutuati dall'Ingegneria del Software, suddivisi nelle seguenti fasi chiave.

2.1. Analisi e studio fattibilità

Il lavoro è iniziato con una approfondita fase di analisi del dominio, finalizzata a circoscrivere il problema originale e a delineare una soluzione tecnologica adeguata. L'obiettivo primario è stato definire un perimetro di progetto che non solo rispettasse rigorosamente i requisiti accademici del corso, ma che risultasse anche realistico e sostenibile in termini di tempistiche ed energie del team, garantendo la consegna di un prodotto concretamente in grado di risolvere la problematica individuata.

2.2. Progettazione e mockup visivo

Per fornire una visione chiara e condivisa dell'applicativo prima ancora di scrivere codice, è stato realizzato un prototipo interattivo (Mockup) utilizzando Figma. Questa fase si è rivelata cruciale per:

- Immaginare l'interfaccia utente (UI) finale e definire la User Experience (UX).
- Identificare in anticipo le funzionalità necessarie e i dati che dovevano essere esposti a video, agevolando l'individuazione di potenziali barriere di accessibilità fin dal

design. Parallelamente, è stata creata una struttura documentale centralizzata. Questa «Source of Truth» conteneva le specifiche di dominio, i diagrammi dei casi d'uso, i diagrammi delle classi e la mappa dei routing web. L'approvazione di questa documentazione ha evitato fraintendimenti e limitato drasticamente la necessità di successivi rimaneggiamenti del codice.

2.3. Organizzazione del lavoro e gestione dei task

La scomposizione del lavoro e il tracciamento delle attività sono stati gestiti tramite l'ecosistema GitHub, sfruttando una Project Board dedicata. Tutte le funzionalità derivanti dall'analisi dei casi d'uso sono state tradotte a priori in specifiche «Issue» architetturali, contenenti il dettaglio delle classi, dei metodi e dei template HTML necessari al loro completamento. Questo approccio di pianificazione «up-front» ha permesso di:

- Stabilire le priorità di implementazione (Core features prima delle funzionalità aggiuntive).
- Parallellizzare lo sviluppo, evitando sovrapposizioni o conflitti sul codice («pestarsi i piedi»).
- Fornire metriche quantitative costanti sull'avanzamento dei lavori, agevolando la ricalibrazione delle energie del team.

2.4. Flusso di lavoro e versioning

Per il controllo di versione è stato adottato un approccio basato sul Feature Branching.

- Ogni Issue è stata sviluppata all'interno di un branch isolato, mantenendo integro il branch principale di sviluppo (develop).
- L'integrazione del nuovo codice in develop avveniva esclusivamente tramite Pull Request. L'obbligo di Code Review da parte di altri membri del team ha garantito un controllo di qualità sistematico, elevando la stabilità e la completezza della baseline condivisa.

2.5. Qualità del Codice e Continuous Integration

La fase di codifica è stata accompagnata dalla scrittura di test automatizzati (Unit Test). L'esecuzione di questi test è stata automatizzata tramite le GitHub Actions al momento della Push/Pull Request. Questo processo di integrazione continua (CI) ha permesso di monitorare il corretto funzionamento dei metodi chiave in modo rapido e autonomo, offrendo la garanzia che l'aggiunta di nuovo codice (o il refactoring di codice esistente) non generasse regressioni su funzionalità già validate. La sintesi di queste metodologie organizzative ha reso la fase di codifica rapida, efficiente e non dispersiva, consentendo di reinvestire le risorse risparmiate in ampie sessioni di testing finale, con particolare focus sulla verifica dell'accessibilità dell'applicativo.

3. Fasi del progetto

3.1. Analisi dei Requisiti

La fase di analisi dei requisiti è stata fondamentale per definire il perimetro del progetto, identificare gli utenti target e stabilire le regole di business del dominio. Attraverso un

processo di elicitazione e affinamento, sono stati definiti gli attori del sistema, i casi d'uso principali e i vincoli di integrità sui dati.

3.1.1. Attori del sistema

Il sistema UniFix prevede interfacce e permessi differenziati per tre tipologie di utenti:

- **Studiante (Utente Base):** È il fruitore principale degli spazi universitari. Può navigare il sistema, cercare aule, monitorarne lo stato e aprire nuove segnalazioni di guasto. Dispone inoltre di un'area personale per salvare le aule tra i preferiti.
- **Tecnico:** È l'operatore incaricato della manutenzione. Ha il compito di supervisionare le segnalazioni aperte, prenderle in carico in totale autonomia e portarle a risoluzione.
- **Amministratore (Admin):** Ha un ruolo di supervisione globale. Gestisce le utenze del sistema (con facoltà di rimozione) e può intervenire in via eccezionale sull'eliminazione delle segnalazioni. Per semplicità progettuale, i profili di livello superiore (Admin e Tecnico) vengono pre-caricati nel sistema (hardcoded), mentre la registrazione è aperta solo agli utenti base.

3.1.2. Casi d'uso principali

1. **Autenticazione:** Registrazione (solo per studenti), Login e Logout.
2. **Esplorazione e Preferiti:** Ricerca testuale delle aule e visualizzazione del dettaglio dell'aula (con indicazione chiara dell'edificio e dell'indirizzo). Gli studenti possono aggiungere o rimuovere le aule da una propria lista di preferiti, visualizzabile direttamente in un carosello nella Home Page.
3. **Apertura Segnalazione:** Creazione di un nuovo ticket di guasto. Per ottimizzare la User Experience, la selezione del luogo del guasto è gestita in modo dinamico lato client (tramite JavaScript): selezionando un edificio, la tendina delle aule si filtra istantaneamente, o viceversa, selezionando un'aula viene auto-compilato l'edificio di appartenenza.
4. **Gestione Interventi:** I tecnici e gli admin accedono a una lista globale delle segnalazioni. Il tecnico può eseguire transizioni di stato del ticket (da «Aperto» a «In lavorazione», fino a «Chiuso»).
5. **Pannello Amministratore:** L'admin può visualizzare la lista completa degli utenti registrati e procedere alla loro rimozione in caso di necessità, oltre a poter forzare l'eliminazione di qualsiasi issue.

3.2. Architettura e Progettazione

La progettazione tecnica di UniFix è stata orientata alla creazione di un sistema robusto, manutenibile e performante. Per dimostrare la padronanza delle architetture web moderne, il sistema è stato sviluppato interamente in PHP nativo (Vanilla PHP), implementando da zero il design pattern Model-View-Controller (MVC), un sistema di routing avanzato e un motore di templating proprietario, senza ricorrere a framework esterni.

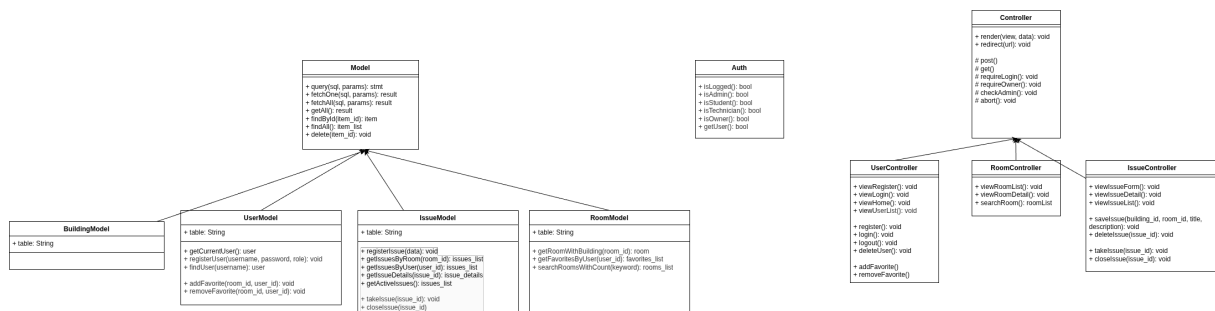
3.2.1. Pattern MVC (Model-View-Controller)

L'applicativo è stato progettato secondo il principio della Separation of Concerns, riflettendosi direttamente nella struttura gerarchica delle directory:

- **Model (src/Models/):** Contiene le classi dedicate all'astrazione del database MySQL. Estendendo una classe base Model.php, i modelli (es. IssueModel, UserModel) incapsulano le query SQL, garantendo la sicurezza contro le SQL Injection tramite l'utilizzo esclusivo di istruzioni preparate (Prepared Statements) fornite dall'estensione PDO.
- **Controller (src/Controllers/):** Ospita la logica di business. I controller (es. RoomController, IssueController) ricevono le richieste dal router, interrogano i Modelli necessari, elaborano i dati formattandoli correttamente per la UI e infine richiamano la View associata, iniettandovi i parametri.
- **View (src/Views/):** La componente visuale è stata suddivisa per favorire il riuso del codice. Le pages rappresentano le singole schermate (es. issueDetailPage.html), i layouts (main.html, auth.html) forniscono la struttura globale ricorrente, mentre i components contengono i frammenti riutilizzabili (es. le card o gli elementi delle liste).

3.2.2. Diagrammi delle classi

A monte della fase di implementazione vera e propria, l'architettura del software è stata rigorosamente modellata attraverso la stesura di diagrammi UML delle classi. Questa fase di progettazione a priori è stata fondamentale per mappare le responsabilità di ogni singolo componente, stabilire le relazioni tra di essi e definire un'interfaccia chiara per i metodi prima ancora di scrivere una singola riga di codice PHP. L'intera organizzazione delle classi sfrutta ampiamente i paradigmi della programmazione orientata agli oggetti (OOP), facendo un uso massiccio dell'ereditarietà per garantire il rispetto del principio DRY e centralizzare la logica di base.



Nello specifico, la gerarchia è stata strutturata come segue:

- **Il layer dei Controller:** È stata definita una superclasse base Controller che implementa tutti i metodi trasversali necessari alla gestione delle richieste HTTP (come il recupero dei parametri get() e post()), le logiche di controllo degli accessi (es. requireLogin(), abort()) e i metodi di output verso la UI (render() e redirect()). Da questa superclasse ereditano i controller specifici di dominio (UserController, RoomController, IssueController), i quali si limitano a orchestrare le chiamate ai modelli e a definire la logica di business specifica per le relative rotte.
- **Il layer dei Modelli:** Analogamente, il livello di accesso ai dati è governato dalla superclasse base Model. Questa classe astratta incapsula la logica complessa di connessione al database (PDO) e fornisce metodi generici per l'esecuzione sicura delle

query (query(), fetchOne(), fetchAll()) e le operazioni standard (come findById() o delete()). Le classi figlie (UserModel, RoomModel, IssueModel, BuildingModel) ereditano questi strumenti e li utilizzano per comporre le query SQL specifiche richieste dai casi d'uso (ad esempio getIssuesByRoom() o addFavorite())

- **Classi di Supporto (Core/Helpers):** Parallelamente alle classi MVC, sono state progettate a monte classi Core per la gestione di servizi trasversali, come la classe Auth, che centralizza la verifica dello stato della sessione e dei privilegi dell'utente (isLoggedIn(), isAdmin(), isTechnician()) in modo sicuro e riutilizzabile da qualsiasi punto dell'applicativo.

3.2.3. Motore di templating

Per mantenere pulite le viste e isolare il PHP dall'HTML (limitando al minimo la presenza di logica condizionale nei file di presentazione), è stata creata la classe core Template.php. Questo mini-motore di templating si occupa di caricare i file HTML e di sostituire specifici placeholder (es. `##ROOM_NAME##`) con i dati reali generati dal Controller, garantendo anche un corretto escaping dei caratteri (htmlspecialchars) per prevenire vulnerabilità XSS.

3.2.4. Routing e sicurezza

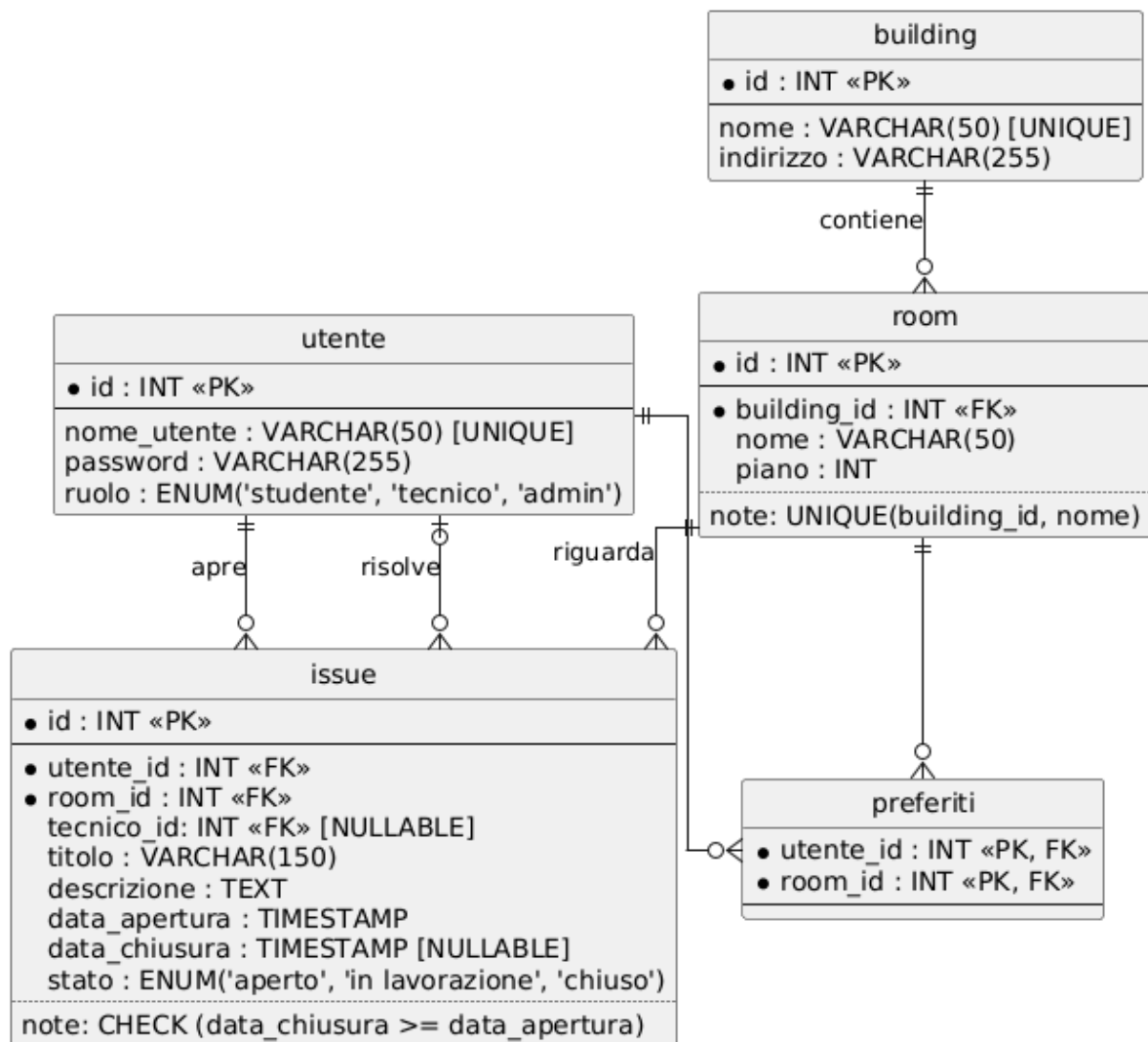
Il punto d'ingresso unico per l'intera applicazione (Front Controller) è il file public/index.php. Questo script inizializza l'ambiente, registra l'autoloader per le classi e passa il controllo alla classe Router.php. Il Router implementa un sistema avanzato di risoluzione delle URL basato su espressioni regolari (regex), permettendo la gestione di rotte dinamiche (es. /issues/{id:num}). Particolarmente rilevante è l'implementazione nativa di un sistema di Middleware per il controllo degli accessi a livello di rotta. Prima di invocare il Controller, il Router verifica stringhe di autorizzazione associate alla singola rotta, quali:

- **guest:** Blocca l'accesso agli utenti già autenticati (es. pagina di login).
- **auth:** Richiede l'autenticazione tramite la classe helper Auth.php.
- **admin / technician:** Verifica il ruolo specifico dell'utente (RBAC - Role Based Access Control).
- **owner:issue:** Esegue un controllo a database per assicurarsi che l'utente che sta tentando un'azione distruttiva (es. l'eliminazione di una segnalazione) sia effettivamente il creatore (owner) della risorsa, a meno che non possieda privilegi di amministratore.

3.2.5. Persistenza dati

3.2.5.1. Progettazione del database e integrità dei dati

La persistenza dei dati è affidata a un database relazionale MySQL. La progettazione dello schema concettuale (modello E/R) e logico è stata guidata dai principi di normalizzazione, con l'obiettivo di garantire la consistenza delle informazioni ed evitare ridondanze.



Il database si articola su cinque entità principali:

- Building e Room: Entità legate da una relazione uno-a-molti (1:N). Un edificio contiene più aule. Per evitare inconsistenze semantiche, è stato applicato un vincolo `UNIQUE(building_id, name)` sulla tabella room, garantendo che non possano esistere due aule con lo stesso nome all'interno del medesimo edificio.
- User: Tabella centralizzata per l'anagrafica, che distingue i privilegi tramite un attributo role di tipo ENUM ("student", "technician", "admin"). Le password sono salvate sotto forma di hash crittografico sicuro.
- Issue: Entità centrale del sistema che traccia le segnalazioni. Mantiene relazioni in chiave esterna con room (l'aula guasta), user (lo studente segnalatore) e, opzionalmente, un secondo user (il tecnico assegnato).
- Favorite: Tabella di mapping per risolvere la relazione molti-a-molti (M:N) tra gli studenti e le aule preferite, avente come chiave primaria composta l'unione di `user_id` e `room_id`.

3.2.5.2. Gestione delle dipendenze e vincoli referenziali

Invece di demandare il controllo delle dipendenze alla logica applicativa (Backend PHP), si è scelto di sfruttare i vincoli nativi del motore relazionale:

- Eliminazione in cascata (ON DELETE CASCADE): L'eliminazione di un edificio (building) comporta l'eliminazione automatica di tutte le sue aule (room). A sua volta, l'eliminazione di un'aula innesca la cancellazione a cascata di tutte le segnalazioni (issue) ad essa collegate e delle relative occorrenze nella tabella favorite.
- Mantenimento dello storico (ON DELETE SET NULL): Qualora un utente (studente o tecnico) venga rimosso dal sistema, i riferimenti testuali all'interno della tabella issue (user_id o technician_id) vengono impostati a NULL anziché eliminare l'intero record. Questo garantisce che lo storico degli interventi e dei guasti dell'Ateneo non venga perso.

3.2.5.3. Vincoli di dominio e logica procedurale

Per garantire che lo stato del sistema rispecchi sempre le regole di business definite in fase di analisi, sono stati implementati:

- Vincoli di Controllo (CHECK): Sulla tabella issue è attivo un vincolo che impedisce l'inserimento di una data di chiusura cronologicamente antecedente alla data di apertura del guasto (CHECK (closed_at IS NULL OR closed_at >= opened_at)).
- Trigger di transizione stato: È stato sviluppato un Trigger a livello di database (trg_remove_technician) che si attiva prima dell'eliminazione (BEFORE DELETE) di un record dalla tabella user. Se l'utente eliminato ha il ruolo di tecnico, il trigger interviene su tutte le segnalazioni a lui assegnate e attualmente «in lavorazione», rimuovendo l'assegnazione (technician_id = NULL) e riportando automaticamente lo stato della segnalazione su «Aperto» (status = "open"). Questo automatismo procedurale garantisce che nessun guasto rimanga «orfano» o bloccato nel sistema a seguito della rimozione del personale

4. Accessibilità

Garantire l'accesso universale alle informazioni è stato un requisito fondante per lo sviluppo di UniFix. L'intero applicativo è stato progettato nel rigoroso rispetto delle direttive WCAG (Web Content Accessibility Guidelines), assicurando che il sistema fosse pienamente fruibile da utenti con disabilità visive, motorie o cognitive, nonché ottimizzato per qualsiasi dispositivo e contesto di fruizione (inclusa la stampa).

4.1. Struttura semantica e supporto agli screen reader

Il codice HTML e CSS è stato sottoposto a validazione rigorosa tramite il Nu Html Checker (W3C) per garantire l'assenza di errori sintattici che potessero compromettere il parsing da parte delle tecnologie assistive. Per facilitare la navigazione non visiva:

- Tag ARIA: È stato fatto un uso mirato degli attributi aria-label per fornire un contesto esplicito agli screen reader laddove il testo visibile non fosse sufficientemente descrittivo (es. bottoni con sole icone o link contestuali come «Visualizza dettaglio: [Titolo Segnalazione]»).

- Navigazione da tastiera: È stato implementato correttamente un link «Salta al contenuto principale» (Skip-link), visibile solo quando riceve il focus, che permette agli utenti che navigano tramite tasto TAB di bypassare la navigazione globale (header/menu) e accedere direttamente al nucleo informativo della pagina.

4.2. Accessibilità visiva e mobile-first

La palette cromatica dell'interfaccia è stata testata per garantire un rapporto di contrasto minimo conforme allo standard WCAG AA tra il testo e lo sfondo. In ottica Mobile-First, la User Experience su smartphone è stata curata nei minimi dettagli:

- La tipografia è gestita con unità di misura relative (rem) per scalare fluidamente.
- Le aree interattive (touch targets) dei pulsanti e dei link sono state dimensionate (tramite adeguati padding) per essere facilmente cliccabili con i polpastrelli, prevenendo tocchi accidentali e frustrazione nell'uso da mobile.

4.3. Visualizzazione dati tabellari

Le tabelle risultano accessibili sia per gli screen reader sia per la fruizione da smartphone:

- Semantica per Screen Reader: Ogni tabella è introdotta da una (visivamente nascosta tramite classe `.visually-hidden`) e da un paragrafo riassuntivo collegato tramite l'attributo `aria-describedby` applicato al tag `<table>`. Questo fornisce subito all'utente non vedente il contesto. Inoltre, l'uso combinato di `<caption>`, `<thead>`, e con gli attributi `scope=«col»` e `scope=«row»` crea una griglia in cui la tecnologia assistiva può sempre associare correttamente il dato alla sua intestazione.
- Linearizzazione Mobile (**Tecnica di Aaron Gustafson**): Aniché ripiegare su uno scroll orizzontale per le tabelle su schermi piccoli, è stata implementata una tecnica CSS avanzata per linearizzare i dati. Su mobile, l'intestazione principale viene nascosta e le singole righe (`<tr>`) diventano elementi a blocco (simili a «card»). Attraverso l'inserimento dell'attributo HTML `data-title` in ogni cella, una media query CSS si occupa di stampare a video il nome della colonna (tramite lo pseudo-elemento `::before` e `content: attr(data-title)`) direttamente prima del dato effettivo. Questo garantisce una lettura verticale comoda, naturale e perfettamente impaginata.

4.4. Stampa delle pagine

Riconoscendo che i documenti amministrativi (come la lista degli interventi) necessitano spesso di essere stampati, è stato redatto un foglio di stile specifico (`@media print`) per ottimizzare la resa su carta, massimizzando la leggibilità e risparmiando inchiostro:

- Tipografia e Colori: Il background viene forzato al bianco e il testo al nero. Il font passa a una famiglia «Serif» (Times New Roman) con dimensione in punti tipografici (12pt), standard ideale per la lettura su carta.
- Pulizia del Layout: Tutti gli elementi puramente interattivi o di navigazione (bottoni, menu, form di ricerca, shadow box, link di skip) vengono rimossi dal flusso della pagina (`display: none`). I layout affiancati vengono forzati a blocco (`display: block`)

- **Esplicitazione dei Link:** Poiché su carta non è possibile cliccare, un'apposita regola CSS (`a[href]::after { content: " (" attr(href) " "; }`) stampa automaticamente l'URL di destinazione di fianco al testo del link. Questa regola è stata disattivata selettivamente per le «breadcrumb» per non generare rumore visivo
- **Gestione Interruzioni di Pagina:** È stata utilizzata la proprietà `page-break-inside: avoid` per impedire che le singole righe delle tabelle vengano tagliate a metà tra due fogli. Inoltre, la regola `display: table-header-group` sul tag assicura che l'intestazione della tabella venga ripetuta automaticamente all'inizio di ogni nuova pagina stampata.

4.5. Accessibilità dei Form e Inserimento Dati

La progettazione dei form di autenticazione e segnalazione ha richiesto un'attenta strutturazione semantica, con l'obiettivo di agevolare l'inserimento dei dati e garantire una navigazione chiara, prevenendo il disorientamento dell'utente. Le soluzioni implementate sono state:

- **Etichettatura esplicita e raggruppamento:** ogni campo di input è associato in modo univoco alla propria etichetta tramite l'attributo `<label for="...">` collegato all'id dell'elemento. Nei form più complessi (come la segnalazione di un guasto), i campi logicamente correlati sono stati racchiusi all'interno di un tag `<fieldset>` descritto da una `<legend>`, fornendo un contesto macroscopico fondamentale per la navigazione sequenziale.
- **Testi di Aiuto e aria-describedby:** Le istruzioni per la compilazione sono state legate al campo di input corrispondente tramite l'attributo `aria-describedby`. In questo modo, quando il campo riceve il focus, lo screen reader legge automaticamente anche la regola di validazione, riducendo il carico cognitivo dell'utente
- **Gestione dei campi obbligatori:** L'obbligatorietà dei campi è delegata all'attributo nativo HTML5 `required`. L'asterisco visivo (*) aggiunto per gli utenti normovedenti è stato isolato dai lettori di schermo utilizzando l'attributo `aria-hidden="true"`, evitando così che il software vocale legga fastidiosamente la parola «asterisco» a ogni campo.
- **Auto-completamento:** Nei form di login e registrazione sono stati utilizzati gli attributi `autocomplete` (`autocomplete="username"`, `autocomplete="new-password"`). Questo facilita il riempimento automatico da parte del browser o dei password manager, un requisito di accessibilità molto importante per agevolare gli utenti.
- **Feedback e Gestione Errori:** I messaggi di errore flash (es. credenziali errate) vengono stampati all'interno di un contenitore dotato dell'attributo `role="alert"`. Questo Live Region ARIA impone alla tecnologia assistiva di interrompere la lettura corrente per notificare immediatamente all'utente l'avvenuto errore, garantendo un feedback tempestivo.

5. Suddivisione dei compiti

Nonostante la natura asincrona del lavoro e l'adozione di una metodologia basata sulla revisione collettiva (Code Review tramite Pull Request), l'architettura modulare del progetto ha permesso di parallelizzare efficacemente lo sviluppo. Ogni membro del team ha preso in carico la responsabilità (ownership) di specifici verticali applicativi o layer strutturali, garantendo un avanzamento costante e ordinato.

- Marco Sanguin:
 - Analisi dei Requisiti: Ha partecipato attivamente alle fasi preliminari del progetto, contribuendo alla definizione, scrittura e affinamento del documento di Use Case (Casi d'Uso) e operando sulle relative Pull Request
 - DevOps e Continuous Integration: Si è focalizzato sul layer di deployment e automazione. Ha configurato la pipeline di Continuous Integration tramite GitHub Actions, automatizzando i processi di test (Smoke Test) e di rilascio del codice sul server (Deploy). Ha inoltre curato asset globali come la configurazione e il routing delle favicon locali
- Enrique Hernandez Gris:
 - Visualizzazione Dati e Navigazione: Ha sviluppato un'ampia porzione delle interfacce di consultazione, occupandosi in modalità full-stack (Model, Controller e View) delle pagine di Lista Aule, Dettaglio Aula, Lista Segnalazioni e Dettaglio Segnalazione
 - Componenti UI Avanzati: Ha implementato la logica di ricerca delle aule, il sistema di navigazione Breadcrumb e ha contribuito in modo significativo all'integrazione della UI per la Dark Mode globale, ottimizzando i colori dei link e la coerenza visiva del tema
- Luca Slongo:
 - Setup e Funzionalità Chiave: Ha avviato il repository curando il setup iniziale della struttura. Ha sviluppato interamente funzionalità core come il motore di aggiunta/rimozione delle aule dai preferiti e le logiche di eliminazione delle segnalazioni (Delete Issue)
 - Refactoring e UI: Si è dedicato a un massiccio lavoro di «finitura» e bug fixing dell'interfaccia utente, risolvendo imperfezioni visive, sistemando i contrasti cromatici, ripulendo l'HTML da attributi non necessari (es. tabindex superflui) e consolidando l'accessibilità generale delle pagine.
- Aldo Bettega:
 - Infrastruttura e Core: Ha definito la progettazione iniziale delle classi e l'infrastruttura MVC di base.
 - Backend e Funzionalità: Si è occupato dello sviluppo dell'autenticazione (Login/Logout), della gestione utenti lato Admin, del form di creazione delle segnalazioni (Issue form) e della logica di gestione stato delle segnalazioni (Take/Close issue). Ha inoltre sviluppato la Home Page personalizzata per ruolo.
 - Accessibilità e Testing: Ha curato le fasi critiche dell'accessibilità, implementando la Dark Mode nativa, linearizzando le tabelle per il mobile, configurando il foglio

di stile per la stampa accessibile e garantendo il superamento dei test del Nu Html Checker. Ha parallelamente implementato gran parte degli Unit Test (PHPUnit). Ha curato la stesura finale della documentazione e della relazione.

6. Conclusioni e Sviluppi futuri

6.1. Conclusioni

Il progetto UniFix ha raggiunto con successo gli obiettivi prefissati in fase di analisi. È stata consegnata una piattaforma web robusta, sicura e pienamente accessibile, capace di snellire e centralizzare la gestione dei guasti all'interno degli spazi universitari.

Lo sviluppo interamente «Vanilla» (senza l'ausilio di framework esterni) ha permesso di consolidare e dimostrare sul campo una profonda comprensione delle architetture web moderne. L'implementazione da zero del design pattern MVC, unita allo sviluppo di un sistema di routing con middleware e alla gestione sicura del database (Prepared Statements, Trigger e vincoli relazionali), testimonia la solidità ingegneristica dell'applicativo. Inoltre, la rigorosa attenzione all'accessibilità (W3C, WCAG AA, navigazione semantica) e l'adozione di un approccio Mobile-First assicurano che il prodotto finale sia un ambiente inclusivo e facilmente fruibile da chiunque.

6.2. Sviluppi futuri

Nonostante il sistema sia completo e funzionale rispetto alle specifiche richieste, l'architettura modulare su cui si fonda si presta a diverse e agili espansioni future, tra cui:

- **Allegati Multimediali:** Estensione del form di segnalazione con la possibilità di caricare file immagine (foto del guasto). Questo agevolerebbe notevolmente la diagnosi a distanza da parte del team di manutenzione.
- **Dashboard Analitica:** Sviluppo di un pannello di controllo avanzato per gli amministratori, dotato di grafici e statistiche (es. edifici con il maggior numero di guasti, tempi medi di presa in carico e risoluzione dei ticket), che risulterebbe uno strumento prezioso per la pianificazione della manutenzione preventiva.